

```
#Import the required packages
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import math
import scipy
```

```
from google.colab import drive
drive.mount('/content/drive');
```

```
#This function to read the dataset collected from sites
#All datasets' attributes are wrote in the excel file Tables.xlsx
#The entire data points are stored in excel files have a name in the last column of Tables file ("Source")
#In case of multiple datasets from the same source they are stored in the same excel file but with different sheets
def read():
```

```
    datasets=pd.read_excel("/content/drive/MyDrive/ScientificProjects/PopulationGrowthModels/Patent/Tables.xlsx")
    print(datasets['Specie']);
    idx=int(input("Enter the index of the required dataset: "))
    #print(idx)
    filename=str(datasets['Source'].iloc[idx])
    #print(filename)
    title=str(datasets['Specie'].iloc[idx])
    #print(title)
    measured_data=pd.read_excel("/content/drive/MyDrive/ScientificProjects/PopulationGrowthModels/Patent/{}.xlsx".format(filename),sheet_name=title)

    return datasets,measured_data,idx,title
```

```
"""This function is to predict a separate growth curve due to the new growth model
    based on entered growth attributes in the files in read function"""
def pred():
```

```
    p_0=float(datasets['P0'].iloc[idx])
    m=float(datasets['m'].iloc[idx])
    p_inf=float(datasets['pinf'].iloc[idx])
    step=float(datasets['step'].iloc[idx])
    k=float(datasets['kexp'].iloc[idx])

    f=p_inf/m
    A=((f*f*m)-(2*f*p_0)+p_0)/((f*f*m*p_0)-(f*f*m*m))
    B=-m*A
    C=((p_0-m)*A)-1/p_0

    p_prev=float()
    t_prev=float()
    dt=float()
    rate=float()
```

```

predicted=[]
predicted.append(p_0)
ts=[]
ts.append(0)

for p in np.linspace(p_0,m,int((m-p_0)/step),True):
    if p==np.linspace(p_0,m,int((m-p_0)/step),True).max():
        #print("For p={}=====".format(p))
        #print("Break!")
        break
    if p==p_0:
        p_prev=p_0
        t_prev=0
        #print("For p0= {}, the t={} ".format(p_prev,t_prev))
    else:
        rate=k*((A*p*p)+(B*p))/(1+(C*p))
        dt=(p-p_prev)/rate
        t_prev+=dt
        ts.append(t_prev)
        p_prev=p
        predicted.append(p)
        #print("For p= {}, t={} ".format(p_prev,t_prev))

"""This part of the fucntion is to reduce the size of the predicted points
that is because of the nature of numerical solutions of differential equations
in which we need a very near points for high accuracy solution, buy be the end
of solution we do not need all these points, a few of them are suffecient
to draw the growth curve"""

t_list=[]
p_list=[]
for i,v in enumerate(measured_data[measured_data.columns[0]]):
    #print("For original data t = {}".format(v))
    for I,V in enumerate(ts):
        if v==V:
            t_list.append(V)
            p_list.append(predicted[I])
            #print("Appended for t = {}".format(v))
            break
        elif V<v:
            t_prev=V
            p_prev=predicted[I]
            continue
        elif V>v:
            t_suc=V
            p_suc=predicted[I]
            t_list.append(V)
            p=p_suc-((t_suc-V)*(p_suc-p_prev)/(t_suc-t_prev))
            p_list.append(p)
            #print("For t = {} appended p = {}".format(v,p))

```

```

        break

try:
    R2=np.corrcoef(list(measured_data[measured_data.columns[1]]), p_list)[0,1]
except ValueError:
    R2=np.corrcoef(list(measured_data[measured_data.columns[1]][:-1]), p_list)[0,1]
#print('R2 = {}'.format(R2))

return p_0,m,p_inf,step,k,f,ts,predicted

```

#This function is to draw the line plot of both actual and predicted growth curves

```

def visualize():

    plt.figure(figsize=(10,6));

    plt.plot(ts,predicted);
    plt.plot(measured_data[measured_data.columns[0]],measured_data[measured_data.columns[
1]],c='#ff7f0e');
    plt.xlabel(measured_data.columns[0])
    plt.ylabel(measured_data.columns[1])
    plt.legend(['Predicted','Original Measured'])
    plt.title(title)

    print("p_0 = {}".format(p_0))
    print("m = {}".format(m))
    print("p_inf = {}".format(p_inf))
    print("k = {}".format(k))
    print("step = {}".format(step))
    print("f = {}".format(f))

```

```

datasets,measured_data,idx,title=read()
p_0,m,p_inf,step,k,f,ts,predicted=pred()
visualize()

```